

GSOI 2026 夏季训练赛

第十三题解

Ehundategh & tfbz

Gioush 大队

2026 年 8 月

目录

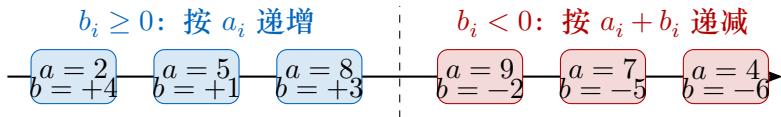
T1 天地皆可往 (roam)

T2 单舟见京杭 (canal)

T3 新水濯旧隍 (moat)

T4 云止聆佳响 (echo)

Analyse



- 会增加龙威的疆土先处理，能够为后续行程留下更多余量。
- 会减少龙威的疆土放在后面，并按照离开时的门槛安排顺序。

数据点 1 ~ 2 10 Pts

- ▶ 固定一个前往疆土的排列后，从前向后模拟即可求出完成这个排列所需的最小初始龙威。
- ▶ 枚举全部 $n!$ 个排列，取这些排列对应答案的最小值。
- ▶ 单次模拟需要 $\mathcal{O}(n)$ 的时间，总时间复杂度为 $\mathcal{O}(n!n)$ 。

数据点 3 ~ 5 15 Pts

- ▶ 二分初始龙威 x 。令 f_S 表示已经前往集合 S 中的疆土时，最多能够剩下多少龙威，不可达状态记为负无穷。
- ▶ 若 $i \notin S$ 且 $f_S \geq a_i$ ，则有

$$f_{S \cup \{i\}} \leftarrow \max\{f_{S \cup \{i\}}, f_S + b_i\}.$$

- ▶ 检查一次需要 $\mathcal{O}(n2^n)$ 的时间。答案具有单调性，因此可以二分最小的可行 x 。

数据点 11 ~ 15 25 Pts

- ▶ 特殊性质 B 保证 $b_i < 0$ 。考虑相邻的两片疆土 i, j 。
- ▶ 若先去 i 再去 j ，除了进入 i 需要满足 $x \geq a_i$ ，进入 j 还需要满足 $x + b_i \geq a_j$ 。
- ▶ 比较两种顺序后可知，应当按照 $a_i + b_i$ 从大到小排列。这个量表示离开疆土 i 时至少能够保留的龙威门槛。
- ▶ 排序后顺序模拟，时间复杂度为 $\mathcal{O}(n \log n)$ 。

数据点 16 ~ 20 25 Pts

- ▶ 先处理所有 $b_i \geq 0$ 的疆土，再处理所有 $b_i < 0$ 的疆土一定不劣。
- ▶ 前一部分按照 a_i 从小到大排列，后一部分按照 $a_i + b_i$ 从大到小排列。
- ▶ 设已经完成的龙威变化量之和为 S 。来到疆土 i 前必须有 $\text{Ans} + S \geq a_i$ ，所以

$$\text{Ans} \leftarrow \max\{\text{Ans}, a_i - S\}, \quad S \leftarrow S + b_i.$$

- ▶ 时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 16 ~ 20 25 Pts

```
1  #include <cstdio>
2  #include <algorithm>
3  #define MAXN 200010
4  using namespace std;
5
6  int c,T,n;
7  struct land{
8      long long Need,Change;
9  }Line[MAXN];
10
11 bool cmp(land A,land B){
12     if((A.Change>=0) != (B.Change>=0)) return A.Change>=0;
13     if(A.Change>=0) return A.Need<B.Need;
14     return A.Need+A.Change>B.Need+B.Change;
15 }
```

数据点 16 ~ 20 25 Pts

```
1 void Solve(){
2     scanf("%d",&n);
3     for(int i=1;i<=n;i++) scanf("%lld%lld",&Line[i].Need,&Line[i].Change
4         );
5     sort(Line+1,Line+n+1,cmp);
6     long long Sum=0,Ans=0;
7     for(int i=1;i<=n;i++){
8         Ans=max(Ans,Line[i].Need-Sum);
9         Sum+=Line[i].Change;
10    }
11    printf("%lld\n",Ans);
12    return ;
13 }
14 int main(){
15     scanf("%d%d",&c,&T);
16     while(T-->0) Solve();
17     return 0;
18 }
```

Contents

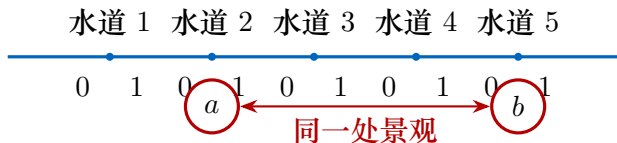
T1 天地皆可往 (roam)

T2 单舟见京杭 (canal)

T3 新水濯旧隍 (moat)

T4 云止聆佳响 (echo)

Analyse



- ▶ 用 (x, a) 与 (y, b) 表示两处重复景观。一次选择记为 C ，其中 $C_i \in \{0, 1\}$ 。
- ▶ 本题中称 C 同时选中重复景观，当且仅当 $C_x = a$ 且 $C_y = b$ 。后文不再使用含义容易混淆的“命中”。

Analyse

选择 C 0 1 0 1 1 0

$$\text{普通总和 } B(C) = \sum_{i=1}^n ((1 - C_i)w_i + C_i v_i)$$

$$\text{实际返回 } \text{Travel}(C) = B(C) - W$$

- ▶ W 是重复景观被重复计算的一份美观度，因此 $W > 0$ 。
- ▶ 因为所有美观度均为正数，所以 $\text{Travel}(C) < B(C)$ 当且仅当 C 同时选中两处重复景观。
- ▶ 这给出一个只依赖返回值的判定函数，之后每次试探都只需判断这个不等式。

数据点 1 15 Pts

- ▶ 当 $n \leq 7$ 时, 可以枚举一位旅客在每段水道选择的方向。
- ▶ 比较 Travel 的返回值与自行计算的 $B(C)$, 即可判断当前选择是否同时选中两处重复景观。
- ▶ 枚举全部 2^n 种选择即可确定答案。

数据点 2 20 Pts

- ▶ 特殊性质 A 保证两处重复景观位于同一侧。
- ▶ 先安排一位旅客全部向左看，再安排一位旅客全部向右看。其中恰有一种选择会同时选中两处重复景观。
- ▶ 固定这份选择。接下来翻转一批位置，便能判断这批位置是否与两处重复景观相交。

数据点 3 40 Pts

	1	2	3	4	5	6	7	8
第 0 位	1	0	1	0	1	0	1	0
第 1 位	0	1	1	0	0	1	1	0
第 2 位	0	0	0	1	1	1	1	0

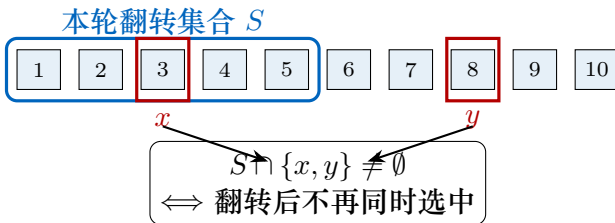
不同编号至少有一位不同

- 若两处景观方向不同，则在能够区分 x, y 的二进制位上，原选择与其补集必有一个同时满足 $C_x = a, C_y = b$ 。

数据点 3 40 Pts

- ▶ 对每个二进制位，令第 i 段水道的方向等于编号 i 的这一位，再询问一次其补集。
- ▶ 对任意两个不同编号，总有一个二进制位能够区分它们。配合原选择与补集，总能得到一份同时选中两处重复景观的选择。
- ▶ 至多使用 $2 + 2\lceil \log_2 n \rceil$ 位旅客就能得到这样一份选择。

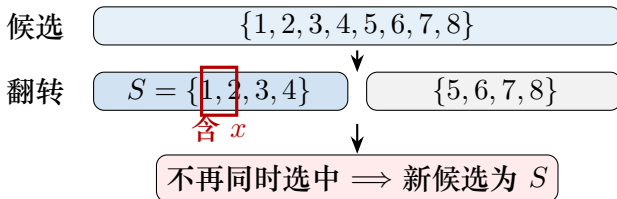
数据点 4 25 Pts



数据点 4 25 Pts

- ▶ 设当前选择 C 已经同时选中两处重复景观。把候选位置的一半记为 S ，同时翻转所有 C_i ($i \in S$)。
- ▶ 翻转后的选择不再同时选中两处重复景观，当且仅当 $S \cap \{x, y\} \neq \emptyset$ 。此时保留 S ，否则保留候选集合的另一半。
- ▶ 不断折半即可找到第一处重复景观。将它从候选集合中删除后重复同样的过程，即可找到第二处。
- ▶ 每次试探后恢复 C 。最后用 Answer 给出两处位置以及 C_x, C_y 。

数据点 4 25 Pts



- 找到第一处后，从候选集合中删去它。此时每次翻转集合都不含第一处，于是同一个判定会精确告诉我们集合中是否含第二处。

正解 100 Pts

- ▶ 先用全左、全右、各二进制位及其补集找到一份同时选中两处重复景观的选择。
- ▶ 再用两次折半定位分别找到两个位置。每次试探后都恢复选择，已经得到的信息始终有效。
- ▶ 总调用次数不超过

$$2 + 2\lceil \log_2 n \rceil + 2\lceil \log_2 n \rceil,$$

在 $n \leq 10^5$ 时不超过 70，可以取得满分。

正解 100 Pts

```
1  #include "canal.h"
2  #include <vector>
3  using namespace std;
4
5  void Init(int c,int T){return;}
6
7  long long Calc(const vector <int> &Choice,const vector <int> &w,const
    vector <int> &v){
8      long long Ret=0;
9      for(int i=1;i<(int)Choice.size();i++) Ret+=Choice[i]?v[i]:w[i];
10     return Ret;
11 }
12
13 bool Judge(vector <int> &Choice,const vector <int> &w,const vector <int>
    &v){return Travel(Choice)<Calc(Choice,w,v);}
```

正解 100 Pts

```
1  int Find(vector <int> &Pos,vector <int> &Choice,const vector <int> &w,  
    const vector <int> &v){  
2      while(Pos.size()>1){  
3          int Mid=Pos.size()>>1;  
4          for(int i=0;i<Mid;i++) Choice[Pos[i]]^=1;  
5          bool Has=!Judge(Choice,w,v);  
6          for(int i=0;i<Mid;i++) Choice[Pos[i]]^=1;  
7          if(Has) Pos.erase(Pos.begin()+Mid,Pos.end());  
8          else Pos.erase(Pos.begin(),Pos.begin()+Mid);  
9      }  
10     return Pos[0];  
11 }
```


正解 100 Pts

```
1 void Locate(int c,int n,vector <int> w,vector <int> v){
2     vector <int> Choice(n+1,0);
3     bool Found=Judge(Choice,w,v);
4     if(!Found){
5         for(int i=1;i<=n;i++) Choice[i]=1;
6         Found=Judge(Choice,w,v);
7     }
8     for(int Bit=0;!Found&&(1<<Bit)<=n;Bit++){
9         for(int i=1;i<=n;i++) Choice[i]=(i>>Bit)&1;
10        Found=Judge(Choice,w,v);
11        if(!Found){
12            for(int i=1;i<=n;i++) Choice[i]^=1;
13            Found=Judge(Choice,w,v);
14        }
15    }
```

正解 100 Pts

```
1  vector <int> Pos;  
2  for(int i=1;i<=n;i++) Pos.push_back(i);  
3  int x=Find(Pos,Choice,w,v),a=Choice[x];  
4  Pos.clear();  
5  for(int i=1;i<=n;i++) if(i!=x) Pos.push_back(i);  
6  int y=Find(Pos,Choice,w,v),b=Choice[y];  
7  Answer(x,a,y,b);  
8  return ;  
9  }
```

Contents

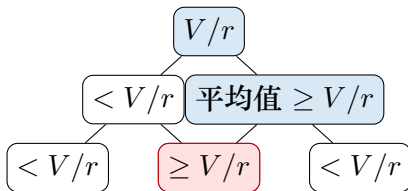
T1 天地皆可往 (roam)

T2 单舟见京杭 (canal)

T3 新水濯旧隍 (moat)

T4 云止聆佳响 (echo)

Analyse



- 当前宫室自身不足平均值时，剩余藏宝量不可能全部落在平均值更低的儿子辖域中。因此至少有一个儿子辖域仍达到目标平均值，沿它下降即可。

数据点 1 15 Pts

- ▶ 当 $n \leq 15$ 时，可以询问所有宫室的转域藏量。
- ▶ 按照深度从大到小处理。宫室 u 自身剩余藏量等于其转域藏量减去所有儿子转域藏量之和。
- ▶ 这样能够还原全部藏量，再直接选择藏量最大的 b 个宫室清洗。

数据点 2 20 Pts

- ▶ 当 $b = 1$ 时，只需找到一间藏量不少于全体平均值的宫室。
- ▶ 从明光殿开始。若当前宫室自身藏量达到平均值就清洗它，否则至少有一个非空儿子转域的平均藏量不小于全局平均值。
- ▶ 向这个儿子继续下降，最终一定能够找到目标宫室。

数据点 3 20 Pts

- ▶ 特殊性质 A 保证王宫是一条链，每个宫室至多只有一个儿子。
- ▶ 询问当前宫室与其儿子的转域藏量，二者之差就是当前宫室自身藏量。
- ▶ 沿链下降即可寻找不低于当前平均值的宫室。每轮需要 $\mathcal{O}(h)$ 次询问。

数据点 4 45 Pts

- ▶ 设当前尚未清洗的宫室数为 r ，剩余藏量和为 V 。本轮寻找一间藏量至少为 V/r 的宫室。
- ▶ 当前宫室的自身藏量可以由当前转域藏量减去两个儿子转域藏量得到。
- ▶ 若自身藏量不足 V/r ，则至少有一个非空儿子转域的平均藏量不小于 V/r ，向该儿子下降即可。

正解 100 Pts

- 清洗一间藏量至少为 V/r 的宫室后，新的剩余藏量至多为

$$V - \frac{V}{r} = V \frac{r-1}{r}.$$

- 连续进行 b 轮后，剩余藏量至多为 $v(n-b)/n$ ，因此取得的藏量至少为 bv/n 。
- 代入评分比例 $p = \min(1, ns/(bv))$ ，得到 $p = 1$ ，所以这是一种对任意合法数据都能取得满分的通用策略。

正解 100 Pts

- ▶ 每个宫室至多有两个儿子。沿一条根到叶路径下降时，每层至多询问两个儿子转域。
- ▶ 一轮的询问次数不超过 $2h + 1$ ，全部 b 轮不超过 $2bh + b$ 。根转域总量可以在开始时询问一次并持续维护。
- ▶ 题目给出的上限为 $2bh + 20$ ，实现时复用已经得到的当前转域藏量即可满足限制。

正解 100 Pts

```
1  #include "moat.h"
2  #include <vector>
3  using namespace std;
4
5  void Init(int c,int T){return;}
6
7  void Play(int c,int n,int b,vector <int> parent){
8      vector <vector <int> > Son(n+1);
9      vector <int> Size(n+1,1),Cleaned(n+1,0),Order;
10     Order.push_back(1);
11     for(int i=2;i<=n;i++) Son[parent[i]].push_back(i),Order.push_back(i);
12     ;
13     for(int i=n-1;i>0;i--) Size[parent[Order[i]]]+=Size[Order[i]];
```

正解 100 Pts

```
1  long long Total=Scout(1);
2  for(int Round=0;Round<b;Round++){
3      long long Current=Total;
4      int Remain=n-Round,Now=1;
5      while(true){
6          vector <long long> Sum(Son[Now].size());
7          long long ChildSum=0;
8          for(int i=0;i<(int)Son[Now].size();i++) Sum[i]=Scout(Son[Now
9      ][i]),ChildSum+=Sum[i];
10         long long Own=Current-ChildSum;
11         if(Own*Remain>=Total) break;
12         int Next=0,NextPos=-1;
13         for(int i=0;i<(int)Son[Now].size();i++){
14             int To=Son[Now][i],Count=Size[To]-Cleaned[To];
```

正解 100 Pts

```
1         if (Count && Sum[i] * Remain >= Total * Count) { Next = To; NextPos = i;  
2         break; }  
3         }  
4         Now = Next; Current = Sum[NextPos];  
5     }  
6     Total -= Clean(Now);  
7     for (int To = Now; To; To = parent[To]) Cleaned[To]++;  
8 }  
9 return ;  
}
```

Contents

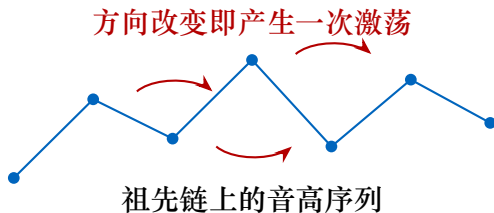
T1 天地皆可往 (roam)

T2 单舟见京杭 (canal)

T3 新水濯旧隍 (moat)

T4 云止聆佳响 (echo)

Analyse



数据点 1 ~ 3 15 Pts

- ▶ 当 $n \leq 20$ 时，可以枚举所有非空听音台集合。
- ▶ 判断这些听音台能否按照祖先关系排成一条龙吟，再统计相邻音高方向改变的次数与鸣响值。
- ▶ 枚举集合并检查，时间复杂度为 $\mathcal{O}(2^n n)$ 。

数据点 4 ~ 7 20 Pts

- ▶ 设 $d_{u,j,0/1}$ 表示以 u 结尾、发生 j 次激荡，最后一段分别上扬或低回的最大鸣响值。
- ▶ 枚举 u 的所有祖先 v 进行转移。若 $a_v < a_u$ ，可以接到上扬状态，否则可以接到低回状态。
- ▶ 每对祖先与后代至多处理 $\mathcal{O}(k)$ 个状态，总时间复杂度为 $\mathcal{O}(n^2k)$ 。

数据点 8 ~ 11 20 Pts

- ▶ 当 $k = 0$ 时，音高方向一旦确定就不能改变。
- ▶ 深度优先搜索时，分别维护各音高下上扬龙吟与低回龙吟的最大鸣响值。
- ▶ 查询严格更低或严格更高音高中的最大值即可完成转移。

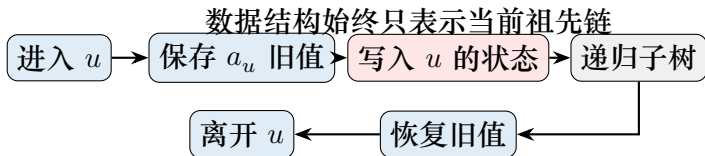
数据点 12 ~ 15 20 Pts

- ▶ 特殊性质 A 保证整棵树是一条链，祖先集合就是当前位置之前的前缀。
- ▶ 对每个激荡次数和末段方向维护一棵按音高建立的线段树。
- ▶ 查询严格更低或严格更高音高的前缀、后缀最大值，时间复杂度为 $\mathcal{O}(nk \log n)$ 。

数据点 16 ~ 20 25 Pts

- ▶ 在一般树上进行深度优先搜索。进入结点 u 时，数据结构中只保留根到 u 的祖先状态。
- ▶ 离开 u 时撤销它对数据结构的修改。这样同一套链上转移可以直接用于一般树。
- ▶ 音高先离散化。每个激荡次数与末段方向各维护一棵线段树。

数据点 16 ~ 20 25 Pts



正解 100 Pts

- ▶ 令 $d_{u,j,0}$ 表示以 u 结尾、发生 j 次激荡且最后一段上扬的最大鸣响值。
- ▶ 令 $d_{u,j,1}$ 表示以 u 结尾、发生 j 次激荡且最后一段低回的最大鸣响值。
- ▶ 单独选择 u 的状态记为 $d_u = w_u$ 。不存在相邻听音台，因此尚未确定末段方向。

正解 100 Pts

- 记 $\max_{a_v < a_u}$ 表示在当前祖先链上对所有满足 $a_v < a_u$ 的状态取最大值，则

$$d_{u,j,0} = w_u + \max_{a_v < a_u} \begin{cases} d_v, \\ d_{v,j,0}, \\ d_{v,j-1,1}, \quad j > 0. \end{cases}$$

- 前两项分别表示从单点开始与继续上扬，第三项表示由低回转为上扬并新增一次激荡。

正解 100 Pts

- 对称地，记 $\max_{a_v > a_u}$ 表示在当前祖先链上对所有满足 $a_v > a_u$ 的状态取最大值，则

$$d_{u,j,1} = w_u + \max_{a_v > a_u} \begin{cases} d_v, \\ d_{v,j,1}, \\ d_{v,j-1,0}, \quad j > 0. \end{cases}$$

- 音高必须严格不同，因此查询区间不包含 a_u 对应的位置。

正解 100 Pts

- ▶ 进入结点前保存各线段树在音高 a_u 处的旧值，写入由 u 得到的新状态。
- ▶ 递归处理完子树后恢复旧值，从而保证兄弟子树之间不会互相转移。
- ▶ 状态数为 $2(k+1)+1$ 。时间复杂度为 $\mathcal{O}(nk \log n)$ ，空间复杂度为 $\mathcal{O}(nk)$ 。

正解 100 Pts

```
1  while(!S.empty()){
2      int Now=S.top().first;
3      bool Exit=S.top().second;
4      S.pop();
5      int Pos=Find(Line[Now]);
6      if(Exit){
7          for(int i=0;i<State;i++) Tree[i].Modify(Pos,Old[Now][i]);
8          continue;
9      }
10     long long Dp[MAXS];
11     for(int i=0;i<State;i++) Dp[i]=INF;
12     Dp[0]=Weight[Now];
```

正解 100 Pts

```
1      for(int j=0;j<=k;j++){
2          int Up=1+(j<<1),Down=Up+1;
3          long long Temp=max(Tree[0].Query(1,Pos-1),Tree[Up].Query(1,
4      Pos-1));
5          if(j) Temp=max(Temp,Tree[1+((j-1)<<1)+1].Query(1,Pos-1));
6          if(Temp>INF/2) Dp[Up]=Temp+Weight[Now];
7          Temp=max(Tree[0].Query(Pos+1,cnt),Tree[Down].Query(Pos+1,cnt
8      ));
9          if(j) Temp=max(Temp,Tree[1+((j-1)<<1)].Query(Pos+1,cnt));
10         if(Temp>INF/2) Dp[Down]=Temp+Weight[Now];
11     }
12     for(int i=0;i<State;i++){
13         Old[Now][i]=Tree[i].Get(Pos);
14         Tree[i].Modify(Pos,max(Old[Now][i],Dp[i]));
15         Ans=max(Ans,Dp[i]);
16     }
```

正解 100 Pts

```
1      S.push({Now,1});
2      for(int i=(int)Son[Now].size()-1;i>=0;i--) S.push({Son[Now][i
3      ],0});
4  }
5  printf("%lld\n",Ans);
6  return ;
7  }
8  int main(){
9      scanf("%d%d",&c,&T);
10     while(T-->0) Solve();
11     return 0;
12 }
```